```
# 2. Removing - be careful with time series data!
data_no_outliers = data[~outliers].copy()

```

When handling outliers in financial data, consider whether they represent legitimate market events before removing them. Some of the most profitable algorithmic trading opportunities occur during outlier events, so automatic removal without understanding the cause could eliminate valuable signals.

Data Normalization and Standardization: Making Comparisons Possible

When working with multiple securities or features with different scales, normalization and standardization make comparisons meaningful:

```
import yfinance as yf
import pandas as pd
import numpy as np

# Download data for two stocks
apple = yf.download("AAPL", start="2023-01-01", end="2023-07-01")['Close']
microsoft = yf.download("MSFT", start="2023-01-01", end="2023-07-01")['Close']

# Combine into one DataFrame
stocks = pd.DataFrame({'AAPL': apple, 'MSFT': microsoft})

# Normalization methods:

# 1. Min-Max Scaling (scale to range 0-1)
min_max_scaled = (stocks - stocks.min()) / (stocks.max() - stocks.min())

# 2. Standardization (z-score: mean=0, std=1)
standardized = (stocks - stocks.mean()) / stocks.std()

# 3. Percent Change from Starting Point (first day = 100)
percent_change = stocks / stocks.iloc[0] * 100

```